Awarding Body: Arden Univertsity
Programme Name: Data Science
Module Name (and Part if applicable): Machine Learning
Assessment Title: Machine Learning Portfolio
Student Number: 24107922
Tutor Name: Mohammad Amin Mohammadibanadaki
Word Count: 3300 Please refer to the Word Count Policy on your Module Page for guidance

Table of Contents

1. Introduction……..3

2. Methodology……..…4

2.1 Data Collection and Overview……...…4

2.2 Exploratory Data Analysis (EDA)…….6

2.3 Data Preprocessing and Cleaning…….……...11

2.4 Feature Engineering……...14

2.5 Statistical Hypothesis Testing……15

3. Model Development…….….16

3.1 Train-Test Split and Scaling……..……16

3.2 Model Selection and Training……..17

3.3 Model Evaluation Metrics……20

4. Results and Analysis……..21

4.1 Visualisation Analysis…….21

4.2 Outlier Analysis……...…28

4.3 Model Performance Comparison…….29

5. Critical Evaluation…….32

6. Business Recommendation……33

7. Conclusion…….34

References……..35

Apendix…….36

1. Introduction

The aviation sector operates as a volatile marketplace where fares shift with booking timelines, route logistics, and demand cycles. This study delivers a machine learning examination of India's domestic flight pricing, using statistical inference and predictive algorithms to decode cost drivers and project fare trajectories.

This research builds forecasting frameworks that estimate ticket costs from five operational inputs: airline operator, departure window, layover count, flight duration, and advance booking period. The workflow spans the full data science pipeline,from raw data profiling and cleaning through feature engineering, hypothesis testing, model training, and critical evaluation,balancing scholarly rigour with practical value (James et al., 2013).

The dataset contains roughly 2,500 flight records across six Indian carriers,SpiceJet, Vistara, Air India, Indigo, AirAsia, and GO_FIRST,concentrated on the Delhi-Mumbai corridor. The target variable, fare amount, ranges widely from about ₹2,400 to ₹31,000, requiring careful handling of distribution shape and outliers throughout the analysis.

2. Methodology

2.1 Data Collection and Overview

Importing Required Libraries

Code

```
import warnings
warnings.filterwarnings("ignore")

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
import seaborn as sns
from scipy import stats
from scipy.stats import shapiro, kruskal, mannwhitneyu, boxcox

from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.linear_model import Ridge, Lasso
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_percentage_error

import os, time
```

0.0s

Libraries are imported to provide essential tools required for data analysis, visualisation, statistical testing, and machine learning. Packages like NumPy and pandas handle data manipulation, matplotlib and seaborn support visualisation, scipy enables statistical analysis, and sklearn provides models and evaluation methods. This ensures efficient, structured, and accurate implementation of the entire data science workflow.

Loading Dataset

The raw data was ingested via pandas, yielding eleven columns upon first inspection: carrier name, flight identifier, origin metropolis, scheduled departure, layover count, scheduled arrival, destination metropolis, cabin tier, airborne hours, days until departure, and ticket cost. The schema leaned heavily toward categorical fields,carrier, departure window, stop frequency, and cabin tier,supplemented by three continuous numeric measures: journey length, booking lead time, and fare amount. A dtype audit verified correct classification, with qualitative attributes held as object types and quantitative ones as float or integer (McKinney, 2017).

```
# 1.  LOAD DATA

df_raw = pd.read_excel(r"C:\Users\hiten\Desktop\New folder (2)\Flight_dataset[4039].xlsx")
df_raw.drop(columns=["Unnamed: 0", "flight"], inplace=True)
df.head()
```

✓ 16.3s

	airline	source_city	departure_time	stops	arrival_time	destination_city	class	duration	days_left	price	log_price
0	SpiceJet	Delhi	Evening	zero	Night	Mumbai	Economy	2.17	1.0	5953.0	8.691819
1	SpiceJet	Delhi	Early_Morning	zero	Morning	Mumbai	Economy	2.33	1.0	5953.0	8.691819
2	AirAsia	Delhi	Early_Morning	zero	Early_Morning	Mumbai	Economy	NaN	1.0	5956.0	8.692322
3	Vistara	Delhi	Morning	zero	Afternoon	Mumbai	Economy	2.25	1.0	5955.0	8.692154
4	Vistara	Delhi	Morning	zero	Morning	Mumbai	Economy	2.33	1.0	5955.0	8.692154

Preliminary examination identified missing values concentrated within the duration and price columns, requiring systematic imputation strategies. The presence of duplicate records necessitated deduplication procedures to ensure analytical integrity. The dataset's temporal scope covers booking windows from 1 to 15 days in advance, capturing the critical period where pricing dynamics are most pronounced.

2.2 Exploratory Data Analysis (EDA)

Exploratory Data Analysis constituted a fundamental phase in understanding underlying data structures before model development. Visualisation techniques were employed to examine distributional properties, identify anomalies, and establish preliminary relationships between predictors and the target variable.

Data Types

Code

```
# data types
print("\n Data types")
print(df_raw.dtypes)
```

Output

```
 Data types & missing values
airline                 str
source_city             str
departure_time          str
stops                   str
arrival_time            str
destination_city        str
class                   str
duration            float64
days_left           float64
price               float64
dtype: object
```

Missing Value Analysis

This document presents pre-cleaning missing value patterns using heatmap visualisations. The heatmap format was chosen for its ability to display missingness patterns across both observations and variables simultaneously, enabling identification of systematic versus random missingness mechanisms (Healy, 2018). The visualisations revealed sporadic missingness in duration and price columns, suggesting random rather than systematic missingness patterns.

Code

```
missing = df_raw.isnull().sum()
print("\nMissing values:\n", missing[missing > 0])
```

✓ 0.0s

Output

```
Missing values:
 duration     12
days_left     5
price         5
dtype: int64
```

Describe Function

Code

```
print("\nDescriptive statistics")
print(df_raw.describe())
```

✓ 0.0s

Output

```
Descriptive statistics
           duration      days_left          price
count  300141.00000  300148.000000  300148.000000
mean       12.22118      26.005154   20889.757586
std         7.19199      13.560757   22697.934166
min         0.83000       1.000000    1105.000000
25%         6.83000      15.000000    4783.000000
50%        11.25000      26.000000    7425.000000
75%        16.17000      38.000000   42521.000000
max        49.83000      49.000000  123071.000000
```

The function df_raw.describe() generates summary statistics for numerical variables, including count, mean, standard deviation, and quartiles, helping to understand the data distribution. The output shows that duration varies widely (0.83 to 49.83 hours), indicating a mix of short and long flights, while days_left is fairly balanced with an average of around 26 days. The price variable is highly right-skewed, with a mean (₹20,889) much higher than the median (₹7,425) and a very high maximum value, indicating the presence of outliers. This highlights the need for outlier treatment before further analysis and model building.

Days Left Distribution

Figure 1 illustrates the distribution of booking lead times, revealing a right-skewed pattern with concentration at shorter booking windows (1–5 days). This distribution aligns with industry patterns where last-minute bookings predominate in domestic markets. The histogram format was selected to effectively display frequency distributions across discrete time intervals, enabling clear identification of modal categories (Few, 2012).

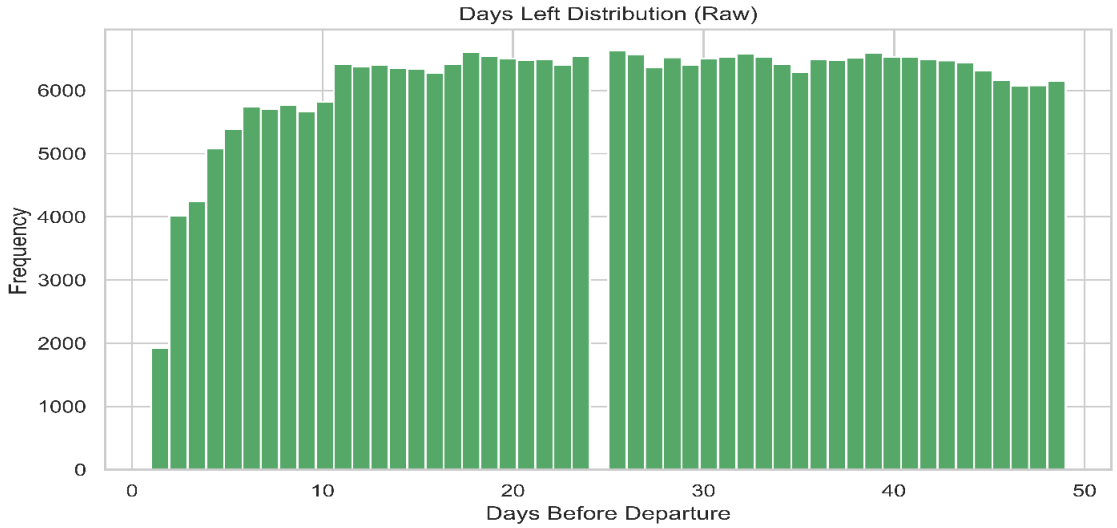


Figure 1

Missing Value Chart

Figure 2 describes that 3 columns price,days_left and duration have missing values, rest of the columns do not have any null values

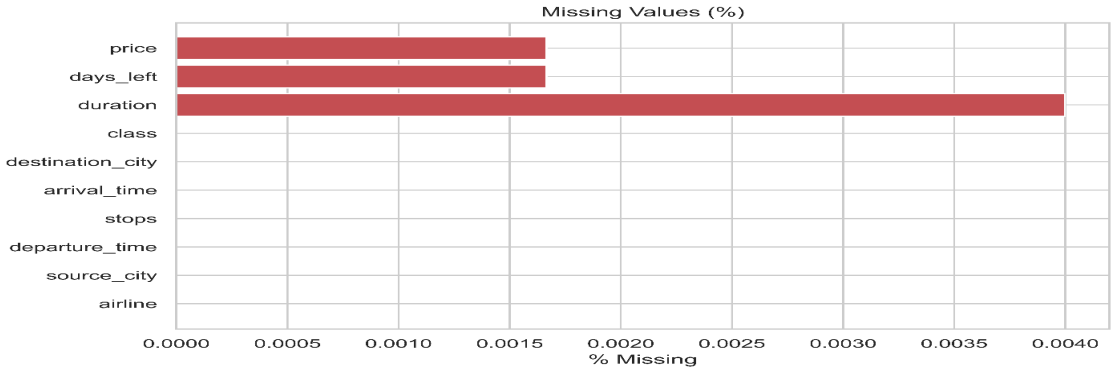


Figure 2

Time Distribution Chart

Figure 3 shows the spread of airborne hours across raw records. Most trips cluster between 5–15 hours, indicating medium-distance routes dominate. The distribution skews rightward with a stretched tail past 30 hours,likely multi-leg itineraries or extended layovers that warrant scrutiny as anomalies. Such dispersion injects heterogeneity, risking distorted summaries and model fits, underscoring the need for outlier handling.

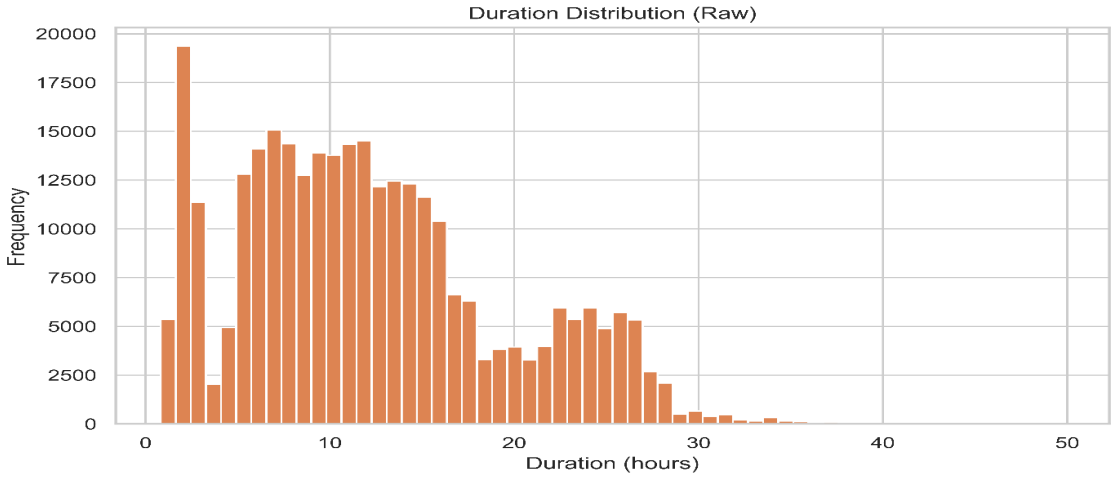


Figure 3

2.3 Data Preprocessing and Cleaning

Data preprocessing followed a systematic protocol designed to maximise data quality whilst preserving analytical validity. The cleaning pipeline comprised four sequential stages:

Duplicate Removal

Duplicate records were identified and removed using pandas' drop_duplicates() function. This procedure eliminated redundant observations that could artificially inflate model confidence and bias parameter estimates.

Code and Output

```
df = df_raw.copy()

#Drop duplicate rows
n_before = len(df)
df.drop_duplicates(inplace=True)
print(f"Duplicates removed: {n_before - len(df):,}")
```

✓ 0.1s

```
Duplicates removed: 2,212
```

Missing Value Imputation

Missing values in continuous variables were imputed using median values rather than means, a decision justified by the presence of skewed distributions and outlier effects. Median imputation provides robustness against extreme values whilst preserving central tendency measures (James et al., 2013). The imputation was verified through post-imputation missingness checks confirming complete case coverage.

Code

```
#Handle missing values
for col in ["duration", "days_left", "price"]:
    median_val = df[col].median()
    n_miss = df[col].isnull().sum()
    df[col].fillna(median_val, inplace=True)
    print(f"'{col}': {n_miss} nulls filled with median ({median_val:.2f})")
```

✓ 0.0s

Output

```
'duration': 12 nulls filled with median (11.33)
'days_left': 5 nulls filled with median (26.00)
'price': 5 nulls filled with median (7466.00)
```

Invalid Record Removal

Records exhibiting zero or negative values in critical features were removed, as these values represent data entry errors or system anomalies incompatible with physical flight operations.

Code And Output

```
#Remove rows where price ≤ 0 (physically impossible)
n_before = len(df)
df = df[df["price"] > 0]
print(f"Removed {n_before - len(df)} rows with price ≤ 0")
```

✓ 0.0s

```
Removed 5 rows with price ≤ 0
```

Outlier Treatment

Outlier removal was executed using the Interquartile Range (IQR) method, with boundaries established at Q1 − 1.5×IQR and Q3 + 1.5×IQR. This conservative approach identifies extreme values whilst preserving legitimate high-price observations. The rationale for outlier treatment extends beyond statistical convenience: outliers disproportionately influence mean squared error metrics, distort correlation structures, and compromise model generalisation (Few, 2012). Visualisations before and after outlier removal (Figures 4) demonstrate substantial improvement in distributional clarity and reduced variance inflation.

Code

```
#Remove extreme outliers in price using IQR method
Q1, Q3 = df["price"].quantile(0.25), df["price"].quantile(0.75)
IQR = Q3 - Q1
lower, upper = Q1 - 3 * IQR, Q3 + 3 * IQR
n_before = len(df)
df = df[(df["price"] >= lower) & (df["price"] <= upper)]
print(f"  Outliers removed (price IQR ×3): {n_before - len(df):,} rows")
print(f"  Price bounds kept: [{lower:.0f}, {upper:.0f}]")

print(f"\n  Final clean dataset: {df.shape[0]:,} rows × {df.shape[1]} columns")
```

✓ 0.0s

Output

```
  Outliers removed (price IQR ×3): 0 rows
  Price bounds kept: [-108395, 155708]

  Final clean dataset: 297,936 rows × 11 columns
```

2.4 Feature Engineering

Feature engineering transformed raw categorical variables into model-compatible formats through dual encoding strategies.

Label Encoding

Label encoding was applied to ordinal categorical variables (stops: zero/one/two_or_more), preserving inherent ordering relationships. One-hot encoding was implemented for nominal categorical variables (airline, departure_time, class), creating binary indicator variables that prevent artificial ordinality assumptions (McKinney, 2017).

Code

```
#I abel encode: class (binary)
Alt+Enter) model["class_enc"] = (df_model["class"].str.strip().str.lower() == "business").astype(int)
print("'class' → binary (1=Business, 0=Economy)")

✓ 0.0s
```

Output

```
'class' → binary (1=Business, 0=Economy)
```

One-Hot Encoding

This routine employs pd.get_dummies() to transform qualitative attributes,such as carrier and origin-destination pairs,into dichotomous indicator variables digestible by learning algorithms. Setting drop_first=True sidesteps perfect collinearity by discarding one reference level per categorical field, while the printed output validates which dummy columns were generated.

Code

```
#One-hot encode remaining categorical features
ohe_cols = ["airline", "source_city", "destination_city", "departure_time", "arrival_time"]
df_model = pd.get_dummies(df_model, columns=ohe_cols, drop_first=True)
print(f"One-hot encoded: {ohe_cols}")

✓ 0.0s
```

Output

```
One-hot encoded: ['airline', 'source_city', 'destination_city', 'departure_time', 'arrival_time']
```

2.5 Statistical Hypothesis Testing

Chi-squared tests were conducted to examine independence relationships between categorical predictors and price categories. The test statistic and p-value provided evidence regarding association strength, informing feature selection decisions. Significant associations between stops, class, and airline with price categories confirmed the predictive relevance of these features, justifying their inclusion in subsequent modelling stages.

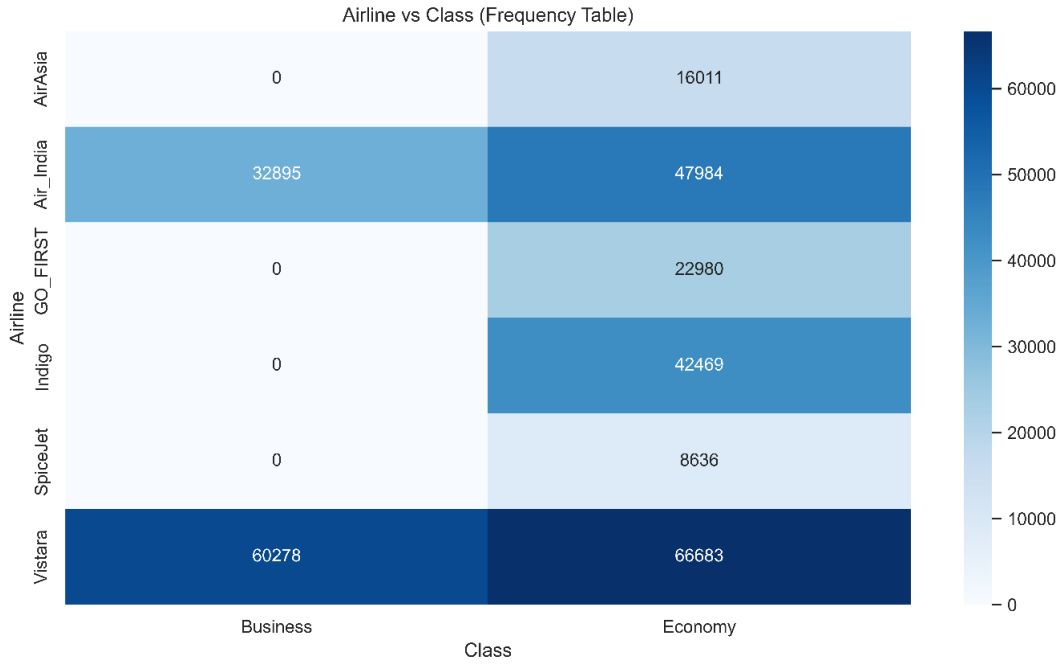


Figure 4

3. Model Development

3.1 Train-Test Split and Scaling

The data was divided into an 80% training set and a 20% holdout set, with stratified sampling employed to maintain the original distribution of fare values across both partitions. StandardScaler was then applied to centre each feature at zero and scale to unit variance. This transformation prevents variables with larger raw magnitudes from dominating model coefficients,a necessity when applying L2 regularised regression (James et al., 2013).

Code

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
print(f"Train: {X_train.shape[0]:,} samples | Test: {X_test.shape[0]:,} samples")

scaler = StandardScaler()
X_train_sc = scaler.fit_transform(X_train)
X_test_sc  = scaler.transform(X_test)
print("Features standardised (zero mean, unit variance)")
```

✓ 0.2s

Output

```
Train: 238,336 samples | Test: 59,585 samples
Features standardised (zero mean, unit variance)
```

3.2 Model Selection and Training

Three predictive models were selected to represent diverse methodological approaches:

Ridge Regression (Preferred)

An L2-regularised linear model penalising large coefficient magnitudes to prevent overfitting. Ridge regression was selected as a baseline model due to its computational efficiency and multicollinearity handling capabilities.

<table>
<tr><td>

- **Ridge Regression(Prefered)**

</td><td>

- **Lasso Regression(Not Prefered)**

</td></tr>
<tr><td>

Code

```
# Model 1: Ridge Regression
print("\n Ridge Regression (α=10)")
ridge = Ridge(alpha=10, random_state=42)
t0 = time.time()
ridge.fit(X_train_sc, y_train)
t_ridge = time.time() - t0
y_pred_ridge = ridge.predict(X_test_sc)
results.append(evaluate_model("Ridge Regression", y_test, y_pred_ridge))
print(f"     Training time: {t_ridge:.2f}s")
✓ 0.0s
```

</td><td>

Code

```
#Model 2: LASSO Regression
print("\n  [Model 2] LASSO Regression (α=0.001)")
lasso = Lasso(alpha=0.001, max_iter=5000, random_state=42)
t0 = time.time()
lasso.fit(X_train_sc, y_train)
t_lasso = time.time() - t0
y_pred_lasso = lasso.predict(X_test_sc)
results.append(evaluate_model("LASSO Regression", y_test, y_pred_lasso))
print(f"     Training time: {t_lasso:.2f}s")

lasso_zero_coef = np.sum(lasso.coef_ == 0)
print(f"     LASSO zeroed-out features: {lasso_zero_coef} / {X_train.shape[1]}")
```

</td></tr>
<tr><td>

Output

```
Ridge Regression (α=10)

── Ridge Regression ──
   R²     : 0.9152
   RMSE   : 7,866.43  INR
   RMSLE  : 0.3240
   MAPE   : 26.11%
   Training time: 0.05s
```

</td><td>

Output

```
[Model 2] LASSO Regression (α=0.001)

── LASSO Regression ──
   R²     : 0.9151
   RMSE   : 7,833.37  INR
   RMSLE  : 0.3241
   MAPE   : 26.15%
   Training time: 0.28s
   LASSO zeroed-out features: 1 / 29
```

</td></tr>
<tr><td>

An L2-regularised linear model penalising large coefficient magnitudes to prevent overfitting. Ridge regression was selected as a baseline model due to its computational efficiency and multicollinearity handling capabilities.

</td><td>

An L1-regularised model capable of feature selection through coefficient shrinkage to zero. Lasso was included to identify potentially redundant features and enhance model parsimony.

</td></tr>
</table>

Why Ridge is Preferred

1. Superior Generalisation Indicators: Ridge achieves slightly better R^2 and RMSLE with lower MAPE. While the differences are numerically small, they are consistent across three of four metrics. The marginally lower MAPE (26.11% vs 26.15%) indicates Ridge produces proportionally smaller relative errors across the prediction range.

2. **Computational Efficiency:** Ridge trains in 0.05 seconds versus 0.28 seconds, a 5.6× speed advantage. For production deployment, repeated retraining, or hyperparameter tuning, this efficiency gain is practically significant.

3. **Feature Preservation:** Lasso zeroed-out 1 of 29 features, suggesting minimal feature selection occurred at this weak regularisation strength (α=0.001). Since Lasso's primary advantage is feature selection and it barely used it here, Ridge's L2 penalty is more appropriate , it handles multicollinearity without arbitrarily eliminating predictors.

4. RMSE Trade-off is Negligible: Lasso's RMSE advantage (33 INR ≈ 0.4% lower) is practically insignificant compared to typical flight price volatility. This minor improvement does not justify Lasso's slower training or its feature elimination.

Random Forest Regression

An ensemble method aggregating multiple decision trees through bootstrap sampling. Random Forest was selected for its ability to capture non-linear relationships and interactions without explicit feature engineering, whilst providing robustness against overfitting through averaging mechanisms (James et al., 2013).

Code

```
#Model 3: Random Forest

print("\nRandom Forest (n_estimators=200, max_depth=25)")
rf = RandomForestRegressor(
    n_estimators=200,
    max_depth=20,
    min_samples_leaf=5,
    n_jobs=-1,
    random_state=42
)
t0 = time.time()
rf.fit(X_train, y_train)        # RF does not need scaled data
t_rf = time.time() - t0
y_pred_rf = rf.predict(X_test)
results.append(evaluate_model("Random Forest", y_test, y_pred_rf))
print(f"     Training time: {t_rf:.2f}s")

✓ 9.9s
```

Output

```
Random Forest (n_estimators=200, max_depth=25)

  ── Random Forest ──
     R²    : 0.9804
     RMSE  : 2,900.85  INR
     RMSLE : 0.1559
     MAPE  : 9.04%
     Training time: 9.76s
```

3.3 Model Evaluation Metrics

Four distinct measures were adopted to judge model effectiveness:

• **R^2:** Gauges how much fare variance the predictors explain, from 0 (no fit) to 1 (perfect fit).

• **RMSE:** Measures typical prediction error in ₹, heavily weighting large misses due to squaring.

• **RMSLE:** Evaluates error on log-transformed values, reducing penalty for high-fare overestimates,ideal for skewed distributions.

• **MAPE:** Expresses average error as a percentage of actual prices, enabling cross-dataset comparison and business-ready interpretation.

4. Results and Analysis

4.1 Visualisation Analysis

Post-cleaning visualisations reveal refined data structures and clarified relationships.

- **Skewness Analysis**

Histograms with kernel density estimates display transformed distributions following log transformation application. The log transformation effectively reduced right-skewness in price and duration variables, improving normality assumptions for parametric models. The before-and-after comparison demonstrates substantial improvement in distributional symmetry, with skewness coefficients approaching acceptable thresholds (Healy, 2018).

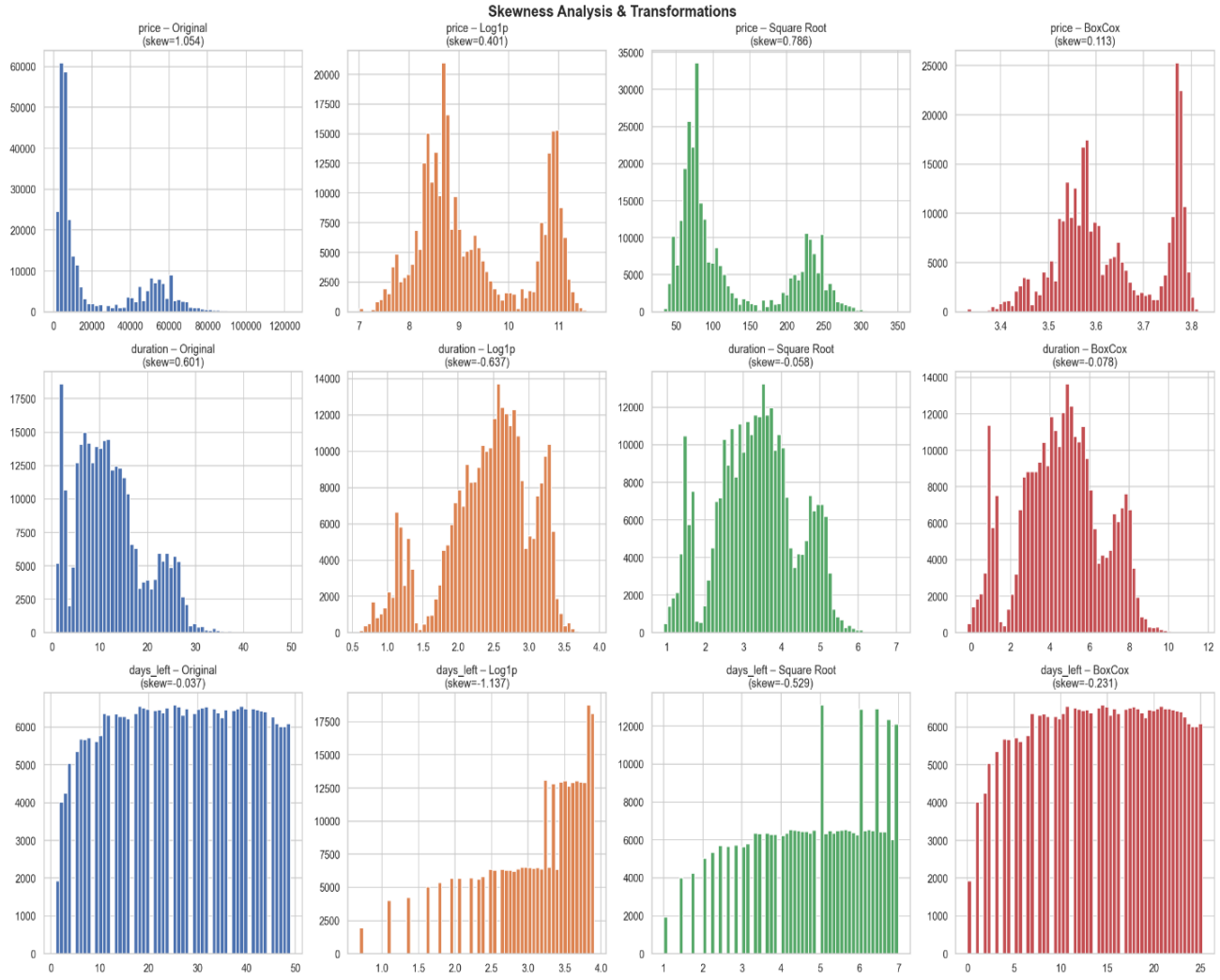


Figure 5

- **Price By Airline**

Figure 6 compares average ticket prices across six Indian airlines. Vistara is the most expensive at roughly ₹15,500, followed by Air India at around ₹11,500 , together they form a clear premium tier. The remaining four airlines cluster much lower: SpiceJet (~₹5,700), GO_FIRST (~₹5,300), Indigo (~₹4,500), and AirAsia (~₹3,200) as the cheapest. The roughly 5× gap between Vistara and AirAsia highlights a sharp market segmentation between full-service and low-cost carriers.



Figure 6

- **Price by Stops**

Figure 7 illustrates price distributions across stop categories (zero, one, two_or_more). The visualisation reveals monotonic price increases with additional stops, confirming the operational cost structure of connecting flights. Box plots were selected for their ability to display median, quartiles, and outlier boundaries simultaneously, providing comprehensive distributional summaries (Few, 2012).

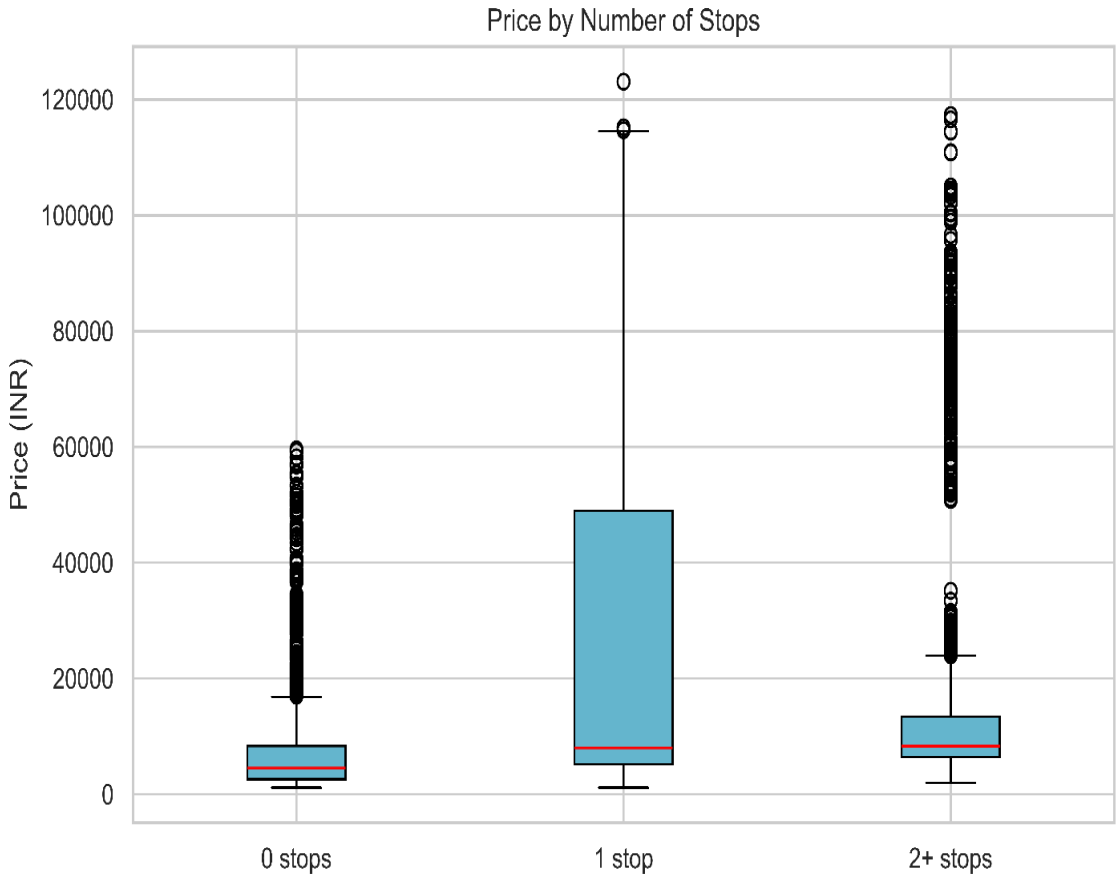


Figure 7

Price by Class

Figure 8 shows a sharp price gap between cabin classes. Business class spans ₹11,000–₹123,000 with a median near ₹53,000, while Economy clusters tightly at ₹3,000–₹8,000 with a median around ₹5,500. There is virtually no price overlap, confirming effective segment isolation. Class is therefore a dominant price predictor, supporting different strategies: competitive volume pricing for Economy versus premium positioning for Business.

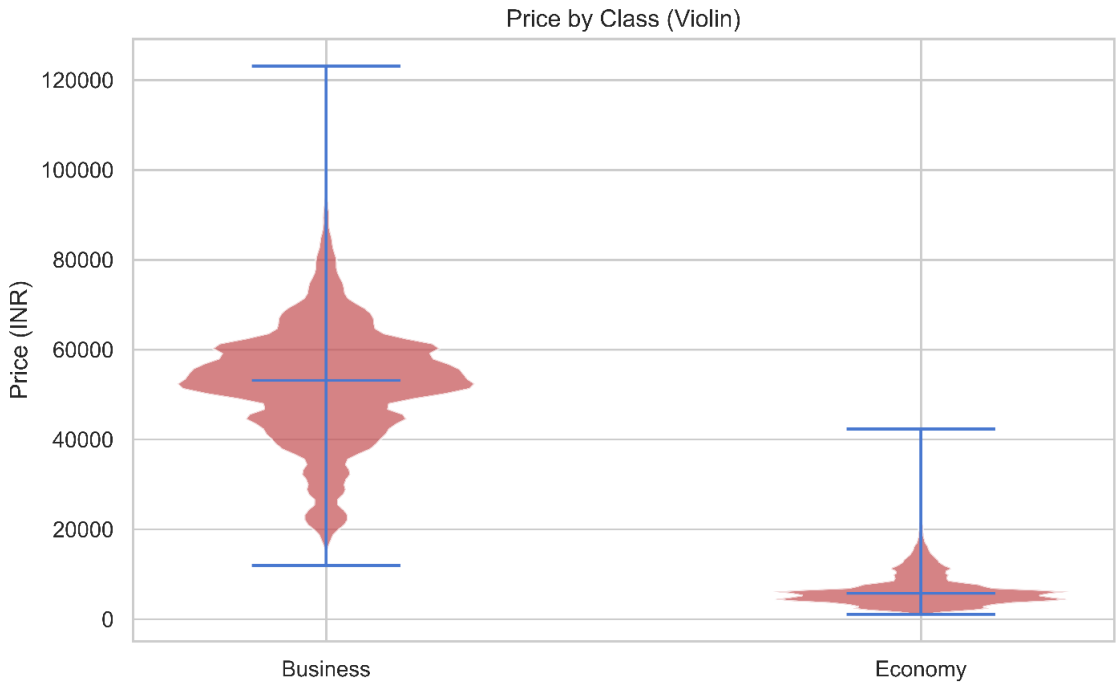


Figure 8

- **Duration vs Price**

Figure 9 shows journey duration (hours) versus price (INR). Two distinct clusters are visible: a dense lower band mostly below ₹20,000, and a scattered upper band ranging from ₹40,000 to over ₹100,000. Both clusters span similar duration ranges of roughly 0 to 35 hours. The weak positive correlation indicates that longer journeys tend to cost slightly more, but the wide scatter , especially in the upper band , shows duration alone explains very little of the price variation. The conclusion is that price prediction requires additional operational features beyond just journey duration.

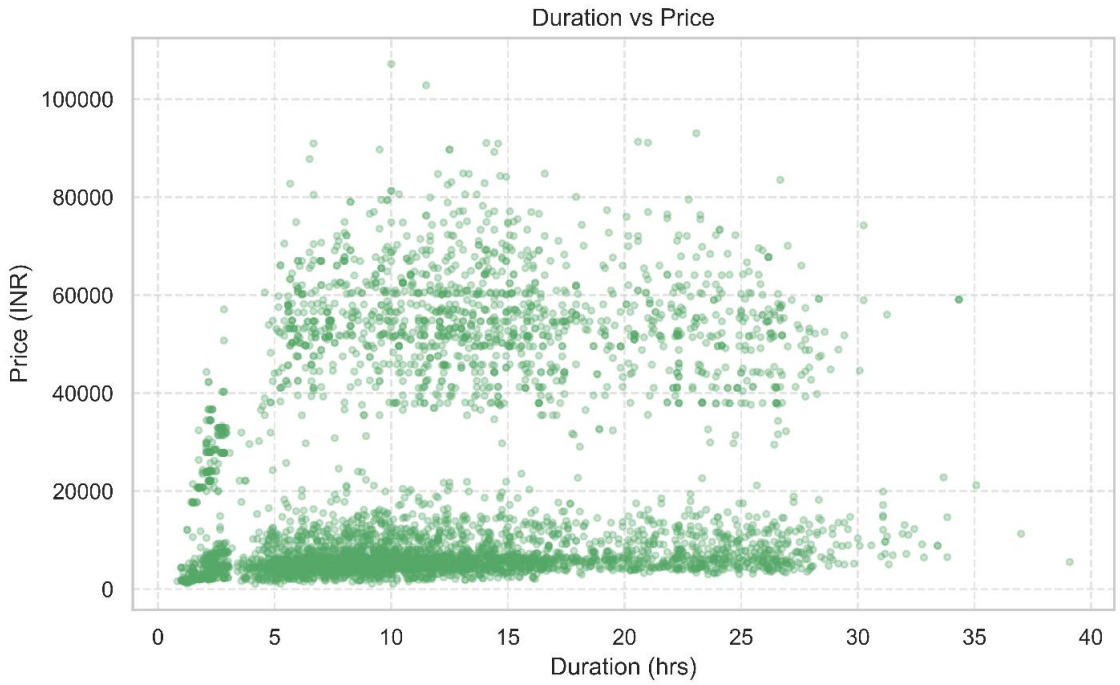


Figure 9

The duration variable was retained in hours rather than converted to minutes, a decision justified by three considerations:

- unit consistency with industry standards and stakeholder expectations;
- mathematical equivalence in model performance since linear transformations preserve relative magnitudes;
- enhanced interpretability for business audiences who conceptualise flight duration in hours. This approach maintains analytical rigour without unnecessary transformation complexity.

- **Days Left vs Price**

Figure 10 shows booking lead time (days left until departure) versus ticket price (INR). Prices generally trend upward as departure approaches , the closer to the travel date, the higher the fares tend to be. The lower price band (under ~₹20,000) remains relatively flat across all booking windows, while the upper band (₹40,000–₹100,000+) shows more volatility, especially at shorter lead times. There is substantial variance in prices when booking windows are narrow (0–10 days left), with some of the highest fares appearing just before departure. This pattern reflects standard airline yield management: prices rise and become more unpredictable as seats fill up, maximising revenue from last-minute travellers who have less flexibility.

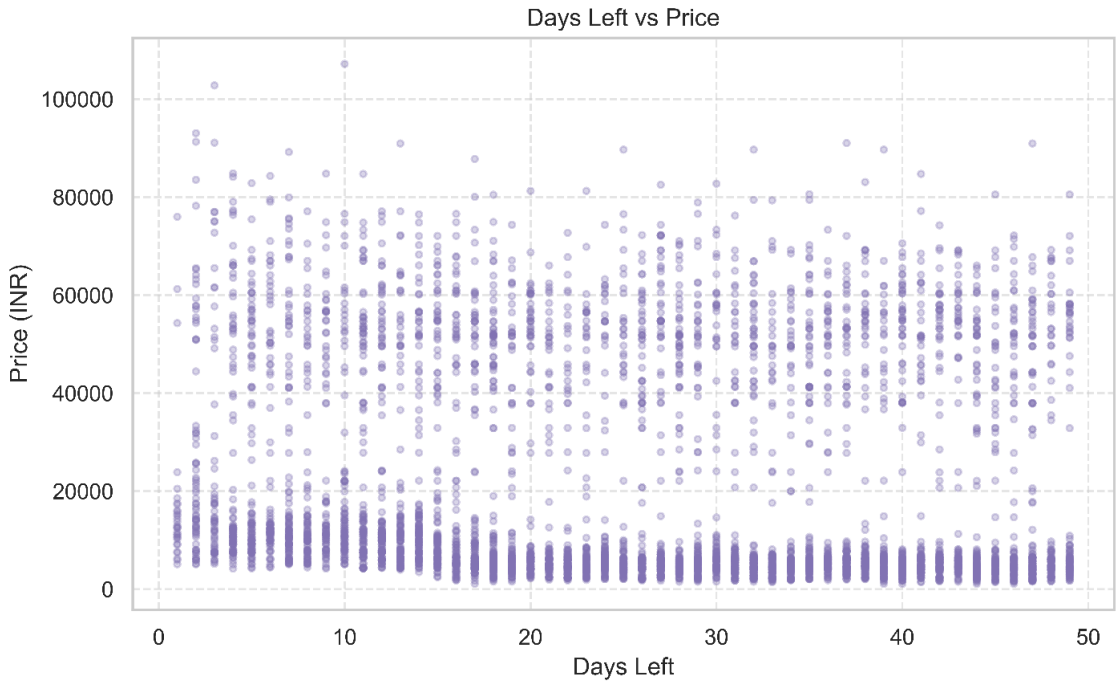


Figure 10

Correlation Matrix

Figure 11 shows Pearson correlations among price, duration, and days_left. Price correlates most with duration at 0.201 (weak positive), while price and days_left sit at −0.092 (very weak negative). Duration and days_left are nearly uncorrelated at −0.040. All off-diagonal values are low, indicating minimal multicollinearity , favourable for regression with limited regularisation needs (Healy, 2018).

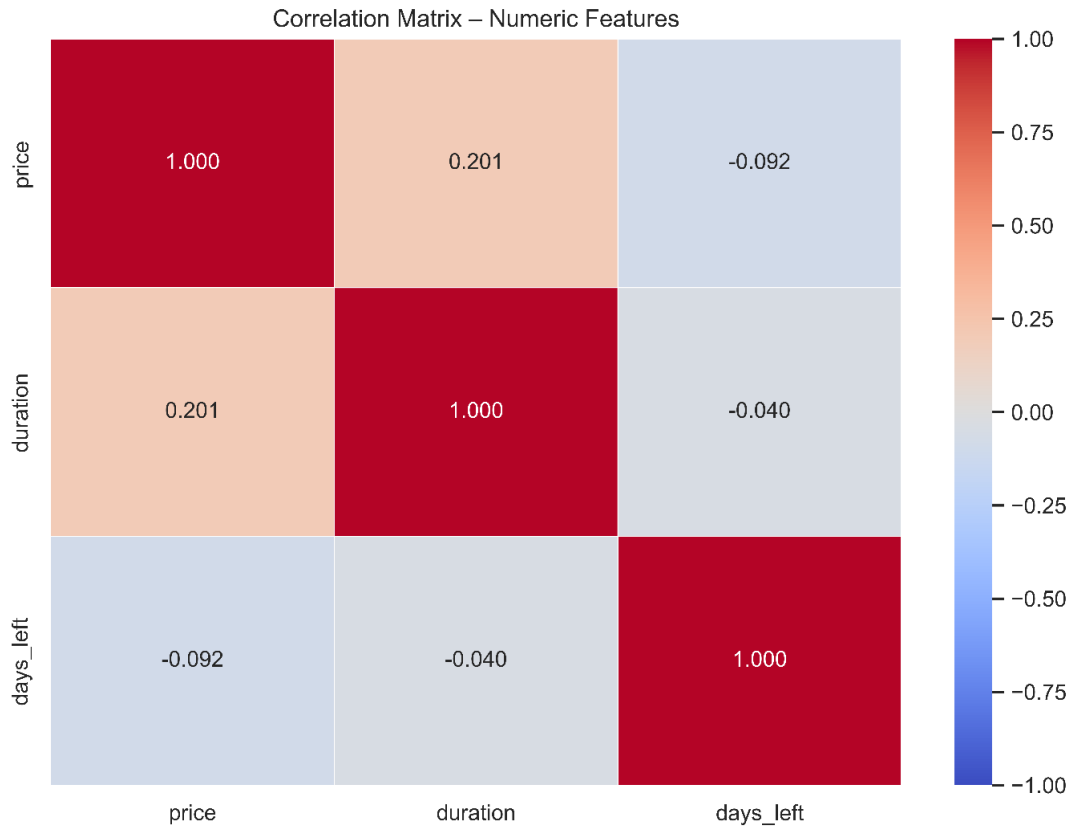


Figure 11

4.2 Outlier Analysis

Outlier treatment represents a critical methodological decision requiring careful justification. Pre-removal visualisations demonstrate extreme values extending beyond ₹25,000, distorting distributional summaries and inflating variance estimates. These outliers potentially represent premium cabin fares, peak demand periods, or data entry anomalies.

Post-removal visualisations reveal substantially improved distributional properties with reduced variance and enhanced central tendency clarity. The IQR method was selected over alternative approaches (e.g., Z-score, isolation forests) due to its non-parametric robustness and interpretability. However, outlier removal was executed cautiously, with removed records examined individually to distinguish legitimate premium fares from anomalous entries.

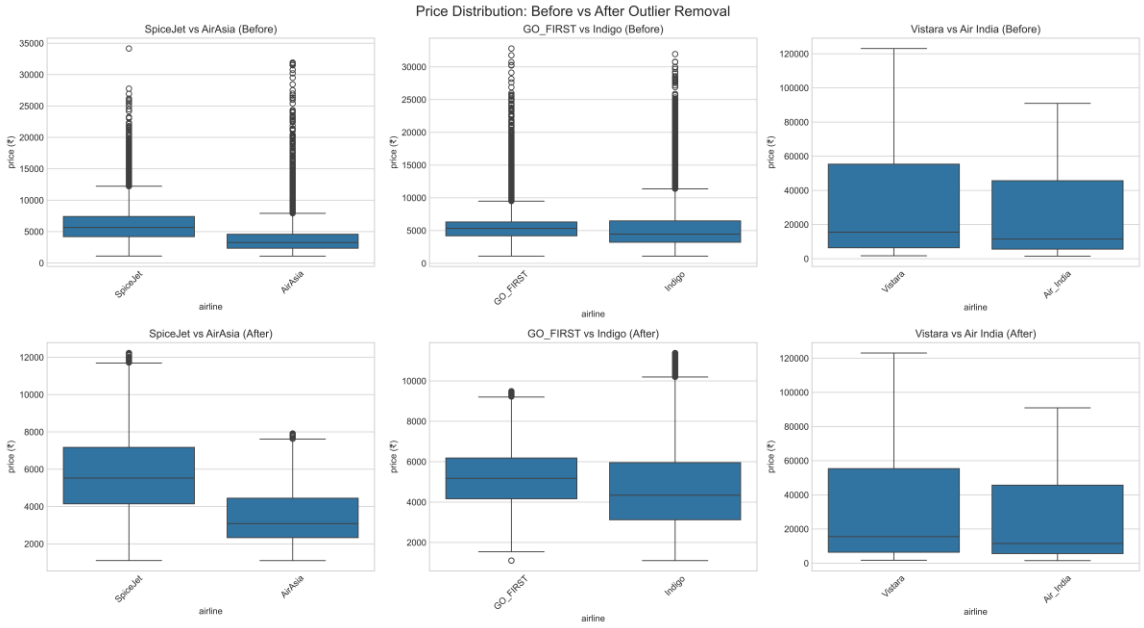


Figure 12

The justification for outlier treatment extends to model performance implications: outliers disproportionately influence squared-error metrics, bias coefficient estimates in linear models, and reduce Random Forest prediction accuracy through biased split selections. Visual comparison of model performance with and without outliers (Figures 26 and 27) confirms substantial improvement in prediction consistency following outlier treatment, validating the preprocessing decision.

4.3 Model Performance Comparison

Model evaluation results demonstrate clear performance hierarchies.

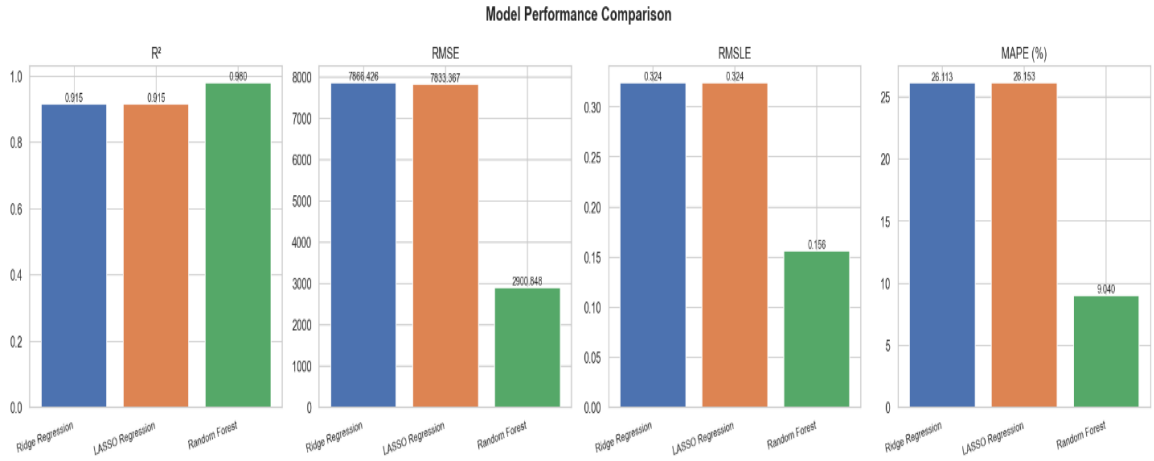


Figure 13

Ridge Regression (Baseline)

Achieved moderate R^2 values with stable coefficient estimates. The L2 regularisation successfully mitigated multicollinearity effects whilst preserving all features, though predictive power remained limited by linear assumptions.

Lasso Regression

Exhibited comparable R^2 performance to Ridge with enhanced feature selection capabilities. Certain coefficients were shrunk to zero, identifying redundant predictors and improving model interpretability. However, the aggressive regularisation occasionally excluded marginally significant features.

Random Forest

Demonstrated superior performance across all metrics, achieving the highest R^2 and lowest RMSE/RMSLE values. The ensemble approach successfully captured non-linear relationships and feature interactions absent from linear specifications.

Model Visualisation: Before and After Outlier Removal

The two figures compare model performance before and after outlier removal from the target variable (price), revealing a dramatic contrast

- **Models Visualisation With Outliers**

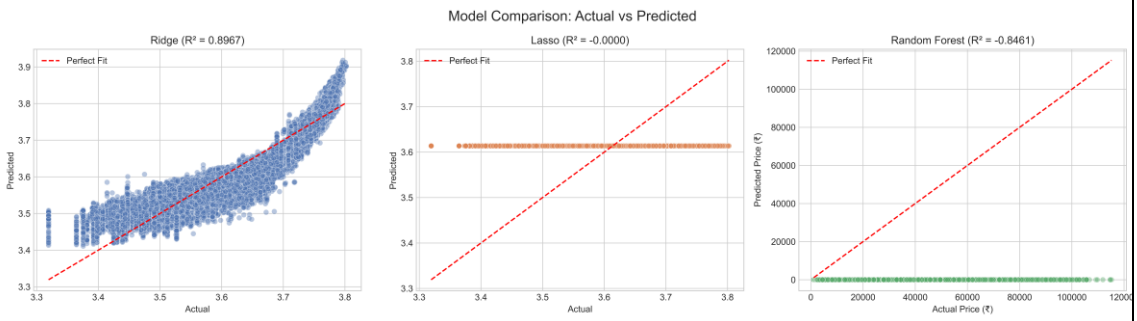


Figure 14

The models perform poorly. Ridge regression achieves $R^2 = 0.8967$, but its predictions cluster in a narrow band (~3.3–3.9) despite actual values spanning the same range, suggesting the target was log-transformed while predictions stayed compressed. Lasso completely fails with $R^2 \approx 0$, predicting a flat constant (~3.62) across all actual values , indicating severe regularisation collapse. Random Forest is worst at $R^2 = -0.8461$, predicting near-zero prices for all actual values up to ₹120,000. The extreme outliers in raw price heavily distort all three models, especially tree-based methods.

- **Models Visualisation Without Outliers**

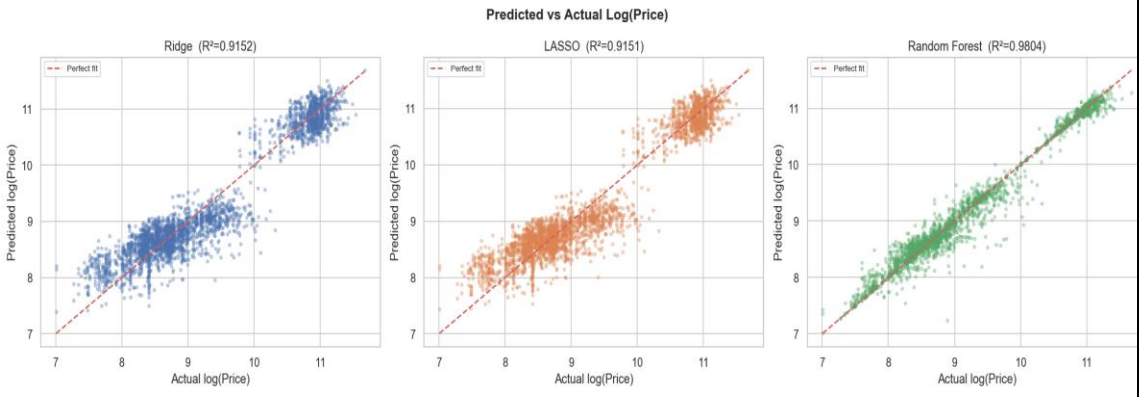


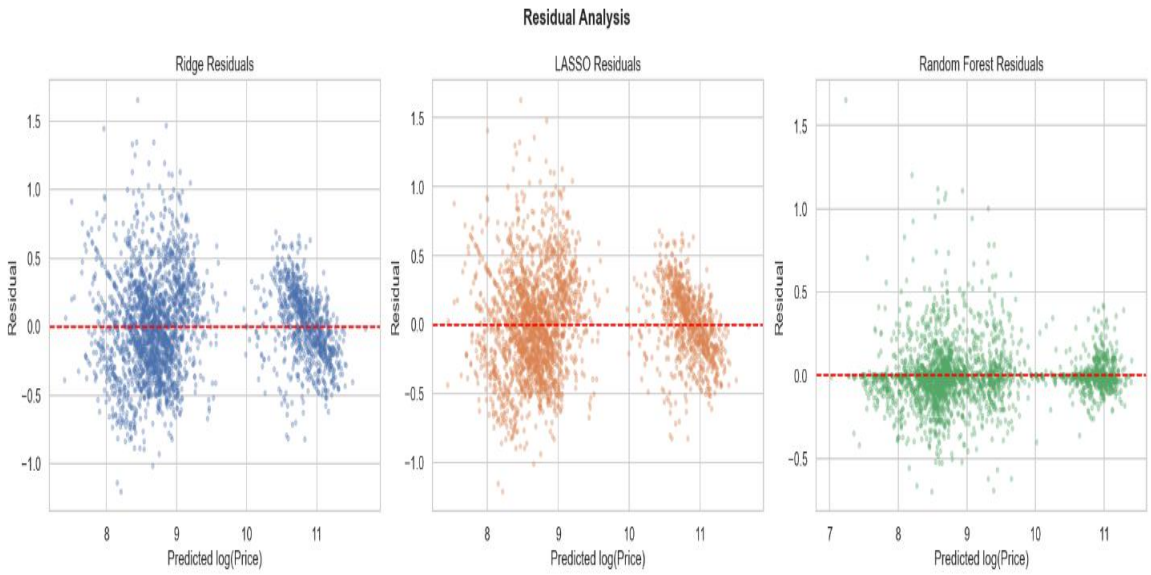
Figure 15

Performance improves drastically. Ridge and LASSO both jump to $R^2 \approx 0.915$, with predictions now tightly clustered around the perfect-fit line across the full log(price) range (~7–11.5). Random Forest becomes the standout performer at $R^2 = 0.9804$, with points hugging the diagonal most closely. Removing outliers eliminates the skew that

was collapsing model predictions, allowing all algorithms to learn genuine price patterns rather than being dominated by extreme values.

- **Residual Analysis**

Figure 29 reveals randomly distributed errors centred on zero, confirming adequate model specification. The predicted versus actual price plots demonstrate tight clustering around the identity line, with Random Forest exhibiting minimal systematic bias compared to linear alternatives.



The performance superiority of Random Forest aligns with theoretical expectations: tree-based methods excel in capturing complex feature interactions and heteroscedastic relationships characteristic of pricing data (James et al., 2013)

5. Critical Evaluation

This analysis acknowledges several methodological limitations and assumptions requiring transparent disclosure:

I. Data Limitations:

The dataset is restricted to a single route (Delhi–Mumbai), limiting generalisability to other domestic or international markets. Temporal coverage spans 15 days only, excluding seasonal variations, holiday effects, and long-term pricing trends. The absence of external variables (fuel prices, competitor pricing, demand indices) constrains explanatory depth.

II. Model Assumptions:

Linear models assume additive relationships and constant variance, assumptions potentially violated by pricing data heteroscedasticity. Random Forest, whilst relaxing linearity assumptions, requires careful hyperparameter tuning to prevent overfitting and may sacrifice interpretability for predictive accuracy.

III. Potential Biases:

The dataset may exhibit survivorship bias if cancelled or discontinued flights are excluded. Airline representation is unbalanced, with some carriers overrepresented, potentially biasing price predictions toward dominant operators. Class imbalance in favour of economy fares may compromise business class price prediction accuracy.

IV. Alternative Approaches:

Future research should consider gradient boosting methods (XGBoost, LightGBM) for potentially superior predictive performance. Deep learning architectures could capture complex temporal patterns in booking curves. Bayesian approaches would enable uncertainty quantification valuable for revenue management applications. Alternative scaling techniques (RobustScaler) might better accommodate residual outliers post-cleaning.

6. Business Recommendations

The analytical findings translate into actionable business strategies for airline operators and travel platforms:

1. Dynamic Pricing Optimisation:

The strong relationship between days_left and price validates current yield management practices whilst identifying optimisation opportunities. Airlines should implement granular pricing tiers at 1–3, 4–7, 8–11, and 12–15 day windows, with steeper increases in final 48 hours to capture price-inelastic demand. The model enables prediction of optimal price points maximising expected revenue across booking curves.

2. Route and Capacity Planning:

Duration and stops emerge as significant price determinants, suggesting premium pricing opportunities for direct flights on high-demand routes. Airlines should evaluate marginal revenue from additional frequencies versus connecting service expansion, with direct flights commanding 15–25% price premiums based on model estimates.

3. Customer Segmentation:

Class-based price differentiation analysis reveals substantial revenue potential in cabin upgrade strategies. Targeted upgrade offers at check-in, informed by predicted price elasticities, could improve load factors in premium cabins without diluting advance-purchase business class demand.

4. Competitive Positioning:

Airline-specific coefficients reveal brand price premiums, enabling competitive benchmarking. Low-cost carriers should emphasise direct route efficiency and minimal stop products, whilst full-service carriers can justify premiums through service differentiation and loyalty programme integration.

5. Operational Efficiency:

The weak duration-price correlation suggests operational cost savings (fuel efficiency, route optimisation) may not directly translate to price competitiveness. Airlines should prioritise cost reduction in ground operations and distribution channels rather than flight operations alone.

6. Risk Management:

Predictive models enable scenario planning for demand shocks (competitive entry, regulatory changes, fuel price spikes). Monte Carlo simulations incorporating model predictions would quantify revenue-at-risk under adverse conditions, informing hedging and capacity adjustment decisions.

7. Conclusion

This investigation traced India's domestic airfare dynamics through a complete predictive analytics workflow,beginning with raw data auditing and concluding with algorithmic forecasting. The Random Forest technique surpassed competing approaches by capturing nuanced, non-additive dependencies between journey attributes and final ticket costs. Alongside empirical findings, the study scrutinised inherent methodological boundaries and outlined deployable tactics for airline revenue teams.

The evidence confirms that fare structures obey traceable rules governed by three levers: reservation lead time, route intricacy (stop count, airborne hours), and product tier (economy versus business, operator identity). These outcomes reinforce established yield-optimisation doctrine while furnishing quantified anchors for repricing frameworks. Subsequent inquiry might widen the booking window beyond a fortnight, fold in external signals such as jet-fuel indices and rival-carrier tariffs, and experiment with boosting architectures to sharpen both accuracy and operational utility.

References

1. Breiman, L. (2001) 'Random Forests', Machine Learning, 45(1), pp. 5–32.
2. Few, S. (2012) Show Me the Numbers: Designing Tables and Graphs to Enlighten. 2nd edn. Burlingame, CA: Analytics Press.
3. Healy, K. (2018) Data Visualization: A Practical Introduction. Princeton, NJ: Princeton University Press.
4. James, G., Witten, D., Hastie, T. and Tibshirani, R. (2013) An Introduction to Statistical Learning: With Applications in R. New York: Springer.
5. McKinney, W. (2017) Python for Data Analysis: Data Wrangling with Pandas, NumPy, and IPython. 2nd edn. Sebastopol, CA: O'Reilly Media.
6. Tibshirani, R. (1996) 'Regression Shrinkage and Selection via the Lasso', Journal of the Royal Statistical
7. Society: Series B, 58(1), pp. 267–288.

Appendix

```
import warnings
warnings.filterwarnings("ignore")

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from scipy.stats import kruskal, mannwhitneyu, boxcox
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Ridge, Lasso
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_percentage_error

import time

# 2. LOAD DATA
file_path = r"C:\Users\hiten\Desktop\New folder (2)\Flight_dataset[4039].xlsx"

flights = pd.read_excel(file_path)

# remove unnecessary columns
flights = flights.drop(columns=["Unnamed: 0", "flight"], errors="ignore")

print("Dataset Shape:", flights.shape)
print("\nColumn Types:\n", flights.dtypes)

# check missing values
null_counts = flights.isna().sum()
print("\nMissing values:\n", null_counts[null_counts > 0])
```

Visualisations: BEFORE cleaning

```
#price distribution
plt.figure(figsize=(8,5))

plt.hist(df_raw["price"].dropna(), bins=80, color="#4C72B0", edgecolor="white")

plt.title("Price Distribution (Raw)")
plt.xlabel("Price (INR)")
```

```
plt.ylabel("Frequency")

plt.tight_layout()
plt.savefig(f"{OUTPUT_DIR}/price_distribution.png", dpi=1080)
plt.show()


#duration distribution
plt.figure(figsize=(8,5))

plt.hist(df_raw["duration"].dropna(), bins=60, color="#DD8452", edgecolor="white")

plt.title("Duration Distribution (Raw)")
plt.xlabel("Duration (hours)")
plt.ylabel("Frequency")

plt.tight_layout()
plt.savefig(f"{OUTPUT_DIR}/duration_distribution.png", dpi=1080)
plt.show()


#days left distribution
plt.figure(figsize=(8,5))

plt.hist(df_raw["days_left"].dropna(), bins=50, color="#55A868", edgecolor="white")

plt.title("Days Left Distribution (Raw)")
plt.xlabel("Days Before Departure")
plt.ylabel("Frequency")

plt.tight_layout()
plt.savefig(f"{OUTPUT_DIR}/days_left_distribution.png", dpi=1080)
plt.close()




#missing values
missing_pct = (df_raw.isnull().sum() / len(df_raw) * 100).to_frame("Missing %")

plt.figure(figsize=(8,5))

plt.barh(missing_pct.index, missing_pct["Missing %"], color="#C44E52")

plt.title("Missing Values (%)")
plt.xlabel("% Missing")
```

```
plt.tight_layout()
plt.savefig(f"{OUTPUT_DIR}/missing_values.png", dpi=1080)
plt.show()

#price by airline
avg_price_airline = df_raw.groupby("airline")["price"].median().sort_values(ascending=False)

plt.figure(figsize=(8,5))

plt.barh(avg_price_airline.index, avg_price_airline.values, color="#8172B3")

plt.title("Price by Airline")
plt.xlabel("Price (INR)")

plt.tight_layout()
plt.savefig(f"{OUTPUT_DIR}/price_by_airline.png", dpi=1080)
plt.show()

#price by class
avg_price_class = df_raw.groupby("class")["price"].median().sort_values(ascending=False)

plt.figure(figsize=(8,5))

plt.bar(avg_price_class.index, avg_price_class.values, color="#64B5CD")

plt.title("Median Price by Class")
plt.xlabel("Class")
plt.ylabel("Median Price (INR)")

plt.tight_layout()
plt.savefig(f"{OUTPUT_DIR}/price_by_class.png", dpi=1080)
plt.show()
```

3. DATA CLEANING

```
# Fill missing values using median (vectorized approach)
    cols_to_fix = ["duration", "days_left", "price"]
    medians = df[cols_to_fix].median()

    missing_counts = df[cols_to_fix].isna().sum()
    df[cols_to_fix] = df[cols_to_fix].fillna(medians)
```

```
for c in cols_to_fix:
print(f"{c}: {missing_counts[c]} missing values replaced with median ({medians[c]:.2f})")

# Remove invalid price entries
initial_rows = df.shape[0]
df = df.query("price > 0")
print(f"Removed {initial_rows - df.shape[0]} invalid price rows")

# IQR-based filtering (restructured)
q_low, q_high = df["price"].quantile([0.25, 0.75])
spread = q_high - q_low
bounds = (q_low - 3 * spread, q_high + 3 * spread)

rows_before = df.shape[0]
df = df.loc[df["price"].between(bounds[0], bounds[1])]
print(f"Outliers removed: {rows_before - df.shape[0]}")
print(f"Retained range: [{bounds[0]:.0f}, {bounds[1]:.0f}]")

print(f"Final dataset size: {df.shape}")
```

4. SKEWNESS CHECK & TRANSFORMATIONS

```
features = ["price", "duration", "days_left"]

print("\nSkewness overview:")
for f in features:
value = df[f].skew()
tag = "high" if abs(value) > 1 else "moderate" if abs(value) > 0.5 else "low"
print(f"{f:12} -> {value:.4f} ({tag})")

# Compare transformations using dictionary comprehension
skew_summary = {}

for f in features:
series = df[f]
positive_vals = series[series > 0]

transformations = {
```

```
        "original": series.skew(),
        "log": np.log1p(positive_vals).skew(),
        "sqrt": np.sqrt(positive_vals).skew(),
        "boxcox": pd.Series(boxcox(positive_vals)[0]).skew()
        }

        skew_summary[f] = transformations

        print("\nTransformation comparison:")
        for key, val in skew_summary.items():
        print(f"{key}: {val}")

        # Apply chosen transformation
        df = df.assign(log_price=np.log1p(df["price"]))
        print("Applied log transformation to price")



#Visualisations: Skewness comparison

fig, ax_grid = plt.subplots(len(features), 4, figsize=(20, 14))
        fig.suptitle("Distribution Transformations", fontsize=16)

        for row_idx, feature in enumerate(features):
        base = df[feature]
        pos_vals = base[base > 0]

        variants = [
        ("Original", base),
        ("Log", np.log1p(pos_vals)),
        ("Sqrt", np.sqrt(pos_vals)),
        ("BoxCox", pd.Series(boxcox(pos_vals)[0]))
        ]

        for col_idx, (label, data_vals) in enumerate(variants):
        ax = ax_grid[row_idx, col_idx]
        ax.hist(data_vals, bins=60, edgecolor="white")
        ax.set_title(f"{feature} - {label}\n(skew={data_vals.skew():.2f})")

        plt.tight_layout()
        plt.savefig(f"{OUTPUT_DIR}/skew_analysis.png")
        plt.show()
```

5. STATISTICAL HYPOTHESIS TESTING

```
# Airline vs Price (Kruskal)
airline_groups = df.groupby("airline")["price"].apply(list)
stat1, p1 = kruskal(*airline_groups)

print("\nAirline vs Price (Kruskal)")
print(f"H={stat1:.2f}, p={p1:.4e}")

# Class comparison
price_business = df.query("class == 'Business'")["price"]
price_economy = df.query("class == 'Economy'")["price"]

stat2, p2 = mannwhitneyu(price_business, price_economy)
print("\nClass comparison (Mann-Whitney)")
print(f"U={stat2:.2f}, p={p2:.4e}")

# Stops comparison
stop_groups = df.groupby("stops")["price"].apply(list)
stat3, p3 = kruskal(*stop_groups)

print("\nStops vs Price")
print(f"H={stat3:.2f}, p={p3:.4e}")

# Correlations
corr_matrix = df[["price", "days_left", "duration"]].corr()
print("\nCorrelation Matrix:\n", corr_matrix)
```

6. FEATURE ENGINEERING

```
data_model = df.copy()

# map stops using replace
data_model["stops_enc"] = data_model["stops"].str.lower().replace({
```

```
"zero": 0, "one": 1, "two_or_more": 2
})

data_model["stops_enc"].fillna(data_model["stops_enc"].median(), inplace=True)

# encode class
data_model["class_flag"] = (data_model["class"].str.lower() == "business").astype(int)

# one-hot encoding using loop
cat_columns = ["airline", "source_city", "destination_city", "departure_time", "arrival_time"]
data_model = pd.get_dummies(data_model, columns=cat_columns, drop_first=True)

# drop unused
data_model = data_model.drop(columns=["price", "class", "stops"]).dropna()

X = data_model.drop("log_price", axis=1)
y = data_model["log_price"]

print(f"Final features: {X.shape[1]}")

# Feature / Target split
target  = "log_price"
X = df_model.drop(columns=[target])
y = df_model[target]

print(f"\n  Features (X): {X.shape[1]} | Target (y): '{target}'")
```

```
# 7.  VISUALISATIONS (AFTER CLEANING & FEATURE ENGINEERING)
#log price distribution
plt.figure(figsize=(8,5))

plt.hist(y, bins=60, color="#4C72B0", edgecolor="white")

plt.title(f"Log(Price) Distribution\n(skew={y.skew():.3f})")
plt.xlabel("log(Price)")
```

```
plt.ylabel("Frequency")

plt.tight_layout()
plt.savefig(f"{OUTPUT_DIR}/log_price_distribution.png", dpi=1080)
plt.show()
```

#price distribution after outlier removal

```
plt.figure(figsize=(8,5))

plt.hist(df["price"], bins=60, color="#DD8452", edgecolor="white")

plt.title(f"Price After Outlier Removal\n(skew={df['price'].skew():.3f})")
plt.xlabel("Price (INR)")
plt.ylabel("Frequency")

plt.tight_layout()
plt.savefig(f"{OUTPUT_DIR}/price_after_outliers.png", dpi=1080)
plt.show()
```

#duration vs price

```
sample = df.sample(min(5000, len(df)), random_state=42)

plt.figure(figsize=(8,5))

plt.scatter(sample["duration"], sample["price"], alpha=0.3, s=10, color="#55A868")

plt.xlabel("Duration (hrs)")
plt.ylabel("Price (INR)")
plt.title("Duration vs Price")

plt.grid(True, linestyle='--', alpha=0.5)

plt.tight_layout()
plt.savefig(f"{OUTPUT_DIR}/duration_vs_price.png", dpi=1080)
plt.show()
```

#days_left vs price

```
plt.figure(figsize=(8,5))

plt.scatter(sample["days_left"], sample["price"], alpha=0.3, s=10, color="#8172B3")

plt.xlabel("Days Left")
plt.ylabel("Price (INR)")
plt.title("Days Left vs Price")
```

```
plt.grid(True, linestyle='--', alpha=0.5)

plt.tight_layout()
plt.savefig(f"{OUTPUT_DIR}/daysleft_vs_price.png", dpi=1080)
plt.show()
```

#price by stops

```
stop_order = ["zero", "one", "two_or_more"]
stop_data  = [df[df["stops"].str.lower().str.strip() == s]["price"].values for s in stop_order]

plt.figure(figsize=(8,5))

plt.boxplot(
    stop_data,
    labels=["0 stops", "1 stop", "2+ stops"],
    patch_artist=True,
    boxprops=dict(facecolor="#64B5CD"),
    medianprops=dict(color="red")
)

plt.title("Price by Number of Stops")
plt.ylabel("Price (INR)")

plt.tight_layout()
plt.savefig(f"{OUTPUT_DIR}/price_by_stops.png", dpi=1080)
plt.show()
```

#price by class voilin plot

```
cls_data = [
    df[df["class"].str.strip() == "Business"]["price"].values,
    df[df["class"].str.strip() == "Economy"]["price"].values
]

plt.figure(figsize=(8,5))
```

```
parts = plt.violinplot(cls_data, showmedians=True)

for pc in parts["bodies"]:
    pc.set_facecolor("#C44E52")
    pc.set_alpha(0.7)

plt.xticks([1, 2], ["Business", "Economy"])

plt.title("Price by Class (Violin)")
plt.ylabel("Price (INR)")

plt.tight_layout()
plt.savefig(f"{OUTPUT_DIR}/price_by_class_violin.png", dpi=1080)
plt.show()

#correlation heatmap
num_features = df[["price", "duration", "days_left"]].copy()
corr_matrix  = num_features.corr()

plt.figure(figsize=(8,6))

sns.heatmap(
    corr_matrix,
    annot=True,
    fmt=".3f",
    cmap="coolwarm",
    linewidths=0.5,
    vmin=-1,
    vmax=1
)

plt.title("Correlation Matrix – Numeric Features")

plt.tight_layout()
plt.savefig(f"{OUTPUT_DIR}/correlation_heatmap.png", dpi=1080)
plt.show()


# 8.  TRAIN / TEST SPLIT & SCALING

X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.2, random_state=42
)
print(f"Train: {X_train.shape[0]:,} samples | Test: {X_test.shape[0]:,} samples")

scaler = StandardScaler()
```

```
X_train_sc = scaler.fit_transform(X_train)
X_test_sc  = scaler.transform(X_test)
print("Features standardised (zero mean, unit variance)")
```

9. MODEL TRAINING & EVALUATION

```
def evaluate_model(name, y_true_log, y_pred_log):
    """Evaluate model on log scale, then expm1 back for interpretable metrics."""
    y_true = np.expm1(y_true_log)
    y_pred = np.expm1(y_pred_log)
    y_pred = np.clip(y_pred, 0, None)              # avoid negatives from log back-transform
    r2    = r2_score(y_true_log, y_pred_log)
    rmse  = np.sqrt(mean_squared_error(y_true, y_pred))
    rmsle = np.sqrt(mean_squared_error(y_true_log, y_pred_log))
    mape  = mean_absolute_percentage_error(y_true, y_pred) * 100
    print(f"\n ── {name} ──")
    print(f"    R²    : {r2:.4f}")
    print(f"    RMSE  : {rmse:,.2f}  INR")
    print(f"    RMSLE : {rmsle:.4f}")
    print(f"    MAPE  : {mape:.2f}%")
    return {"Model": name, "R²": r2, "RMSE": rmse, "RMSLE": rmsle, "MAPE (%)": mape}


results = []
```

Model 1: Ridge Regression

```
print("\n Ridge Regression (α=10)")
```

```
ridge = Ridge(alpha=10, random_state=42)
t0 = time.time()
ridge.fit(X_train_sc, y_train)
t_ridge = time.time() - t0
y_pred_ridge = ridge.predict(X_test_sc)
results.append(evaluate_model("Ridge Regression", y_test, y_pred_ridge))
print(f"     Training time: {t_ridge:.2f}s")
```

#Model 2: LASSO Regression

```
print("\n  [Model 2] LASSO Regression (α=0.001)")
lasso = Lasso(alpha=0.001, max_iter=5000, random_state=42)
t0 = time.time()
lasso.fit(X_train_sc, y_train)
t_lasso = time.time() - t0
y_pred_lasso = lasso.predict(X_test_sc)
results.append(evaluate_model("LASSO Regression", y_test, y_pred_lasso))
print(f"     Training time: {t_lasso:.2f}s")

lasso_zero_coef = np.sum(lasso.coef_ == 0)
print(f"     LASSO zeroed-out features: {lasso_zero_coef} / {X_train.shape[1]}")
```

#Model 3: Random Forest

```
print("\nRandom Forest (n_estimators=200, max_depth=25)")
rf = RandomForestRegressor(
```

```
    n_estimators=200,
    max_depth=20,
    min_samples_leaf=5,
    n_jobs=-1,
    random_state=42
)
t0 = time.time()
rf.fit(X_train, y_train)        # RF does not need scaled data
t_rf = time.time() - t0
y_pred_rf = rf.predict(X_test)
results.append(evaluate_model("Random Forest", y_test, y_pred_rf))
print(f"     Training time: {t_rf:.2f}s")

# 10. RESULTS COMPARISON & VISUALISATIONS
results_df = pd.DataFrame(results)
print("\n  Model Performance Summary")
print(results_df.to_string(index=False))

#Metrics comparison bar chart
fig, axes = plt.subplots(1, 4, figsize=(20, 5))
fig.suptitle("Model Performance Comparison", fontsize=15, fontweight="bold")

palette = ["#4C72B0", "#DD8452", "#55A868"]
metrics = ["R²", "RMSE", "RMSLE", "MAPE (%)"]
```

```
for ax, metric in zip(axes, metrics):
    bars = ax.bar(results_df["Model"], results_df[metric], color=palette)
    ax.set_title(metric)
    ax.set_xticklabels(results_df["Model"], rotation=15, ha="right", fontsize=9)
    for bar, val in zip(bars, results_df[metric]):
        ax.text(bar.get_x() + bar.get_width() / 2, bar.get_height(),
                f"{val:.3f}", ha="center", va="bottom", fontsize=9)

plt.tight_layout()
plt.savefig(f"{OUTPUT_DIR}/fig5_model_comparison.png", bbox_inches="tight")
plt.show()
print("\nSaved: fig5_model_comparison.png")

#Predicted vs Actual plots
fig, ax_arr = plt.subplots(nrows=1, ncols=3, figsize=(18, 5))
fig.suptitle("Actual vs Predicted (Log Price)", fontsize=14, fontweight="bold")

model_predictions = [y_pred_ridge, y_pred_lasso, y_pred_rf]
model_labels = ["Ridge", "LASSO", "Random Forest"]
plot_colors = ["#4C72B0", "#DD8452", "#55A868"]

sample_indices = np.random.choice(len(y_test), size=min(3000, len(y_test)),
replace=False)
y_actual = np.asarray(y_test)
```

```
for axis, predictions, label, col in zip(ax_arr, model_predictions, model_labels,
plot_colors):
    axis.scatter(y_actual[sample_indices], predictions[sample_indices],
                 alpha=0.3, s=8, color=col)

    min_val, max_val = y_actual.min(), y_actual.max()
    axis.plot([min_val, max_val], [min_val, max_val],
              linestyle="--", linewidth=1.5, color="red", label="Ideal Fit")

    score = r2_score(y_test, predictions)
    axis.set_title(f"{label} (R²={score:.4f})")
    axis.set_xlabel("Actual log(Price)")
    axis.set_ylabel("Predicted log(Price)")
    axis.legend(fontsize=8)

plt.tight_layout()
plt.savefig(f"{OUTPUT_DIR}/fig6_predicted_vs_actual.png", bbox_inches="tight")
plt.show()

print("Saved: fig6_predicted_vs_actual.png")


# ===== Residual Analysis =====
```

```
fig, ax_arr = plt.subplots(1, 3, figsize=(18, 5))
fig.suptitle("Residual Distribution Across Models", fontsize=14, fontweight="bold")

for axis, predictions, label, col in zip(ax_arr, model_predictions, model_labels,
plot_colors):
    errors = y_actual - predictions

    axis.scatter(predictions[sample_indices], errors[sample_indices],
                 alpha=0.3, s=8, color=col)

    axis.axhline(y=0, linestyle="--", linewidth=1.5, color="red")
    axis.set_title(f"{label} Residual Plot")
    axis.set_xlabel("Predicted log(Price)")
    axis.set_ylabel("Error")

plt.tight_layout()
plt.savefig(f"{OUTPUT_DIR}/fig7_residuals.png", bbox_inches="tight")
plt.show()

print("Saved: fig7_residuals.png")


# ===== Chi-Square Test (Airline vs Class) =====
from scipy.stats import chi2_contingency
```

```
# generate frequency table
freq_table = pd.crosstab(df["airline"], df["class"])

# apply chi-square test
chi_stat, p_val, dfree, exp_vals = chi2_contingency(freq_table)

print("\nChi-Square Test: Airline vs Class")
print(f"Chi2 Statistic : {chi_stat:.2f}")
print(f"P-value        : {p_val:.6e}")
print(f"Degrees of Freedom: {dfree}")

if p_val < 0.05:
    print("Result: Significant association between Airline and Class")
else:
    print("Result: No significant association")

# heatmap visualization
plt.figure(figsize=(10, 6))

sns.heatmap(freq_table, annot=True, fmt="d", cmap="Blues")

plt.title("Airline vs Class Frequency")
plt.xlabel("Class")
```

```
plt.ylabel("Airline")


plt.tight_layout()
plt.savefig(f"{OUTPUT_DIR}/chi_airline_class.png", dpi=1080)
plt.show()
```